React / Next.js — Complete Learning Roadmap v4.1

> **Gap-Sealed Edition · Full-time learner · 6–8 hrs/day · 22 projects across 4 phases**

How to Use This Roadmap

1. Read the warm-up for each project — ask your AI to drill you on those topics first.
2. Write your architecture plan before opening the code editor.
3. Build until it works. Ugly and working beats pretty and broken.
4. Log the project in the table at the back when done.
5. Move on — do not polish endlessly.

> ⚠️ **Rule:** The warm-up is not optional. If you skip it you will get stuck and not know why.

Before You Start — Non-Negotiables

Git from Day One

Every project = its own folder with its own commits. Commit after every meaningful step.

Format: `feat: add filter` / `fix: count bug` / `refactor: extract hook`

Minimum **6 commits per project**. Never one giant commit at the end.

Folder Structure (use for every project)

```
/app
/components
/lib
/hooks
/types
/context
```

Environment Variables

Secrets go in `.env.local` — never committed. Add to `.gitignore` on day one.

Variables starting with `NEXT_PUBLIC_` are visible to browsers — **never put secrets there.**

How to Unblock Yourself

20 min solo → rubber duck it → search exact error message → ask for help. Never sit stuck silently for hours.

Time Expectations at 6–8 hrs/day

Phase	Time per project
Phase 1	2–4 days each
Phase 2	3–5 days each
Phase 3	5–8 days each
Phase 4	7–10 days each

These are guides, not deadlines.

Phase 1 — Core React Thinking

> 🎯 **Goal:** Components · State · Props · Forms · Derived data · Custom hooks · TypeScript intro
> 🛠️ **Stack:** Next.js App Router — add `"use client"` at the top of every component in Phase 1.
> 🔑 **Key Shift:** Do not overthink server vs client yet — that comes in Phase 2. TypeScript from Project 3 onwards.

Project 1 — Task Manager

> ⏱ *Estimated time: 2 days* · Phase 1

Create, complete, and delete personal tasks. Filter by status (all / active / done).

⚡ **New Concepts:** `useState`, CRUD operations, derived state, basic component tree, controlled inputs

🟣 Warm-up

Topic	What to practise
React components & JSX	Write functional components that accept props and return JSX. Build a `Card` component that takes `title` and `description` as props and renders them.
`useState` — reading and updating	Counter, toggle, text input with live preview. Understand the difference between state and a regular variable.
Controlled inputs	Bind an input's `value` to state and update it with `onChange`. Form with 3 fields, log all values on submit.

Topic	What to practise
Derived state	Given a list of tasks, calculate count of completed ones without a separate `useState`. Filter an array by condition and render the result.
Rendering lists with `.map()` and key props	Render a hardcoded array of 5 items. Add and remove items using state and re-render.

✅ Requirements

- [ ] Add a task with a text input
- [ ] Mark a task as done (toggle)
- [ ] Delete a task
- [ ] Filter by: all, active, done
- [ ] Show count of remaining tasks
- [ ] Accessibility: all buttons must have descriptive `aria-label` attributes
- [ ] Git: minimum 6 commits across the project

✦ Stretch Goal

Persist tasks in `localStorage` so they survive a page refresh.

✓ Ready When

- You can explain why `filteredTasks` is derived state and not its own `useState`
- You can explain the difference between controlled and uncontrolled inputs

---

Project 2 — Expense Tracker

> ⏱ *Estimated time: 3 days* · Phase 1

Log expenses by category. See totals per category and a grand total.

⚡ **New Concepts:** `useReducer`, complex forms with multiple fields, `Object.entries`, array `reduce`

🟣 Warm-up

Topic	What to practise
`useReducer` — when and why	Rewrite a `useState` counter as a `useReducer`. Understand: state, action, dispatch, reducer. Build a shopping cart with add/remove/clear using `useReducer`.
Complex forms (multiple fields)	Form with name, amount, and category all stored in one state object. Log full object on submit and reset after.

Topic	What to practise
Array methods — reduce, filter, find	Use `reduce()` to calculate totals. Group by category into an object using `reduce()`.
`Object.entries` and derived data	Convert `{ food: 45, transport: 20 }` into a rendered list using `Object.entries().map()`.
Form labels and accessibility	Every input must have a `<label htmlFor>` matching the input `id`. 3-field form fully navigable by keyboard.

✅ Requirements

- [ ] Add an expense: name, amount, category
- [ ] Delete an expense
- [ ] Filter by category
- [ ] Show total per category and grand total (derived — never stored in state)
- [ ] At least 4 categories: food, transport, shopping, other
- [ ] Every form input must have a proper `<label>` element

✦ Stretch Goal

Build a simple bar chart using plain CSS (no library) — bar width proportional to amount.

✓ Ready When

- You can explain why you chose `useReducer` over `useState` for this project
- You can explain the difference between a reducer, an action, and dispatch

---

Project 2.5 — Weather Widget ⭐ *Gap-Filler*

> ⏱ *Estimated time: 2 days* · Phase 1

> **Why this exists:** `useEffect` and data fetching must be understood before Project 3. This gap-filler teaches it in isolation before it appears inside bigger projects.

Fetch live weather from a public API and display it.

⚡ **New Concepts:** `useEffect`, `fetch` API, `async/await`, loading states, error states, API keys in `.env.local`

🟣 Warm-up

Topic	What to practise
`useEffect` — what it	Log a message on mount, on every render, and only when a specific state

Topic	What to practise
does and when it runs	changes. Understand the dependency array. Know why `[]` means "run once on mount".
Fetching data with `fetch()` and `async/await`	Fetch a joke from `https://official-joke-api.appspot.com/random_joke` and display it. Reload on button click.
Loading and error states	Show "Loading…" while fetching, error message if fetch fails. Simulate a failed fetch with a bad URL.
Environment variables in Next.js	Store a fake API key in `.env.local`, read it in a server component via `process.env`, confirm it does NOT appear in client bundle.

✅ Requirements

- [ ] Search for a city by name
- [ ] Fetch current weather from Open-Meteo API (free, no key) or WeatherAPI (free tier)
- [ ] Show: temperature, condition, humidity, wind speed
- [ ] Show a loading skeleton while fetching
- [ ] Show a user-friendly error if city not found or fetch fails
- [ ] Store any API key in `.env.local` — never hardcoded

✦ Stretch Goal

Add a 5-day forecast section below the current weather.

✓ Ready When

- You can explain what the `useEffect` dependency array controls
- You can explain the difference between a loading state and an error state

---

Project 3 — Quiz App *(TypeScript starts here)*

> ⏱ *Estimated time: 3 days* · Phase 1

A multi-step quiz — one question at a time, score tracking, results screen.

⚡ **New Concepts:** Multi-step UI state, conditional rendering, component composition, TypeScript interfaces

🟣 Warm-up

Topic	What to practise
TypeScript basics —	Define a `Person` interface with `name: string, age: number`. Create an

Topic	What to practise
types and interfaces	array of `Person` objects. Write a function that takes a `Person` and returns a greeting string.
Typing React props	Add explicit prop types to 3 components. Define a `ButtonProps` interface. Try passing the wrong type and read the error.
Typing `useState`	`useState<string>('')`, `useState<number>(0)`, `useState<Question|null>(null)`. Create a typed quiz question array.
Multi-step UI as a state machine	Build a 3-screen app (start / playing / results) where a `currentScreen` state variable controls which screen is shown.
Conditional rendering patterns	`if/else` JSX, ternary in JSX, `&&` short-circuit. Build a component showing different content based on a `status` prop with 3 possible values.

⚠️ TypeScript Requirements

- Define a `Question` interface: `id`, `text`, `options: string[]`, `correctIndex: number`
- Type all component props — **no `any` allowed anywhere**
- Type your `useState` hooks: `useState<Question[]>([])`, `useState<number>(0)`

✅ Requirements

- [ ] At least 8 hardcoded questions with 4 options each
- [ ] Show one question at a time with a progress indicator (e.g. "Question 3 of 8")
- [ ] Track score
- [ ] Results screen: show score, percentage, and option to restart
- [ ] No going back to previous questions
- [ ] Progress indicator uses `aria-label` or `role="status"` for accessibility

✦ Stretch Goal

Add a countdown timer per question — if time runs out, mark wrong and advance.

✓ Ready When

- You can explain how you managed which screen to show using a single state variable
- You can explain what a TypeScript interface is and why it catches bugs early

---

Project 4 — Recipe Book

> ⏱ *Estimated time: 3 days* · Phase 1

Save and browse recipes with ingredients and steps. Search by name.

⚡ **New Concepts:** Complex nested props, `useRef` for uncontrolled search, custom hooks, TypeScript generics intro

🟣 Warm-up

Topic	What to practise
`useRef` — DOM and non-DOM uses	Access a DOM input with `useRef` and read its value on button click. Also store a **previous state value** in a ref — understand that changing a ref does NOT cause a re-render. Build a search bar that only filters on click, not on every keystroke.
Nested TypeScript types	Define an `Ingredient` type (name, amount, unit). Define a `Recipe` type containing `ingredients: Ingredient[]`. Create 3 recipe objects satisfying the type.
Custom hooks — extracting stateful logic	Move localStorage read/write into a `useLocalStorage(key, initialValue)` hook. Use it in 2 different components with different keys.
Component composition	Build a `Card` component that wraps `children`. Build a `Modal` with a trigger and content. Compose 3 components without prop drilling.
TypeScript generics (basic)	Write `identity<T>(value: T): T`. Then write `useLocalStorage<T>` where T is the type of the stored value.

✅ Requirements

- [ ] List of recipes with name, description, and image (placeholder is fine)
- [ ] Click a recipe to see full details: ingredients + steps
- [ ] Search bar filters by name — use `useRef`, not state, on every keystroke
- [ ] Add a new recipe via a form
- [ ] Recipes persist in localStorage via your custom hook

✦ Stretch Goal

Add a shopping list — select multiple recipes and combine their ingredients, removing duplicates.

✓ Ready When

- You can explain when to use `useRef` instead of `useState` for an input
- You can explain what a custom hook is and give an example of when to extract one
- You can explain a use of `useRef` that does **not** involve accessing the DOM

Project 5 — Kanban Board

> ⏱ *Estimated time: 3 days* · Phase 1

A board with three columns (To Do / In Progress / Done). Move cards between columns.

⚡ **New Concepts:** `useReducer` with complex state shape, shared state across siblings, discriminated union actions

🟣 Warm-up

Topic	What to practise
`useReducer` with complex state	Model a board as `{ columns: { [id]: { title, cards: Card[] } } }`. Write reducer cases for `ADD_CARD`, `DELETE_CARD`, `MOVE_CARD`.
Discriminated union types for actions	Define `type BoardAction = | { type: 'ADD_CARD'; columnId: string; text: string } | { type: 'DELETE_CARD'; cardId: string } | { type: 'MOVE_CARD'; ... }`. Understand how TypeScript narrows the type inside each case.
Lifting state and passing callbacks	Build a parent with state, pass a setter callback to two children, have each child call it.
Immutable state updates	Update a nested object in state WITHOUT mutating it. Add a new card without using `.push()`. Use spread and map.

✅ Requirements

- [ ] Three columns: To Do, In Progress, Done
- [ ] Add a card to any column via an inline form
- [ ] Move a card to the next column with a button
- [ ] Delete a card
- [ ] Show card count per column in the column header

✦ Stretch Goal

Add drag and drop using the native HTML5 drag API (no library).

✓ Ready When

- You can explain why all board state lives in one place and flows down as props
- You can explain what a discriminated union is and why it makes reducers safer

---

Project 5.5 — Theme & Settings App ⭐ *Gap-Filler (NEW)*

> ⏱ *Estimated time: 2 days* · Phase 1

> **Why this exists:** The `useContext + useReducer` combined pattern appears in the tech checklist but is never explicitly built anywhere in the original roadmap. This closes that gap before Phase 2.

A small app where global settings (theme, language, notifications) are managed via `useContext + useReducer` — the most common scalable state pattern in real products.

⚡ **New Concepts:** `useContext + useReducer` combined pattern, `React.createContext`, context provider, consuming context with `useContext`

🟣 Warm-up

Topic	What to practise
`React.createContext` and `useContext`	Create a `ThemeContext`. Wrap your app in a provider. Consume the context value in a deeply nested component without passing any props.
Why `useReducer` inside a context provider	Understand why combining `useReducer` (for complex updates) with Context (for global access) beats passing dispatch through 4 levels of props. Build a counter using this pattern.
Context performance — avoid unnecessary re-renders	Understand that every consumer of a context re-renders when context value changes. Learn to split contexts by domain (e.g. `ThemeContext` vs `UserContext`) to limit blast radius.
TypeScript — typing context values	Type your context: `React.createContext<{ state: AppState; dispatch: React.Dispatch<AppAction> } | undefined>(undefined)`. Handle the `undefined` case safely.

✅ Requirements

- [ ] A global `SettingsContext` built with `useContext + useReducer`
- [ ] Settings: theme (light/dark), language (English/Spanish), notifications (on/off)
- [ ] At least 3 deeply nested components that consume the context without prop drilling
- [ ] A Settings panel that dispatches actions to update state
- [ ] TypeScript: context value fully typed, no `any`
- [ ] Split into at least 2 separate contexts (e.g. `ThemeContext` and `UserContext`) and explain why

✦ Stretch Goal

Add a "reset to defaults" action and an undo/redo mechanism using a history array in the reducer.

✓ Ready When

- You can build the `useContext + useReducer` pattern from memory
- You can explain when NOT to use context (local state that does not need to be global)
- You can explain the performance concern with context and how to mitigate it

---

Project 5C — Habit Tracker 🔁 *Phase 1 Consolidation*

> ⏱ *Estimated time: 3 days* · Phase 1

Track daily habits across a week. Uses every Phase 1 concept together before moving to Next.js.

⚡ **New Concepts:** ALL Phase 1 concepts together — `useState`, `useReducer`, `useRef`, `useEffect`, `useContext`, custom hooks, TypeScript, derived state, localStorage

Self-Check Before Starting

> Answer these without looking:
>
> - When do you use `useReducer` instead of `useState`?
> - When do you use `useRef` instead of `useState`?
> - What is derived state?
> - What does `useEffect`'s dependency array control?
> - When do you use `useContext` instead of prop drilling?
>
> If you cannot answer all five, revisit the relevant projects first.

✅ Requirements

- [ ] Add and delete habits (name, target days per week)
- [ ] Check off a habit for today
- [ ] View a 7-day grid showing completion per day per habit
- [ ] Show a streak count per habit (derived — never stored)
- [ ] Global settings (e.g. week start day) managed via `useContext + useReducer`
- [ ] Persist everything in localStorage via your `useLocalStorage` hook
- [ ] TypeScript throughout — no `any`
- [ ] At least 2 custom hooks extracted
- [ ] Accessible: keyboard navigable, labels on all inputs

✦ Stretch Goal

Add a weekly summary screen showing completion rates as a percentage per habit.

✓**Ready When**

- You can build a multi-feature React app confidently without looking things up
- You are ready for Phase 2 — Next.js, routing, and server components

---

Phase 2 — Next.js Frontend

> 🎯 **Goal:** App Router · Layouts · Server components · Tailwind · Real UI patterns · Libraries
> 🛠️ **Stack:** Next.js App Router, Tailwind CSS, TypeScript throughout.
> 🔑 **Key Shift:** You now think about *where* code runs (server vs client), not just what it does. Libraries are now fair game.

Project 6 — Personal Portfolio Site

> ⏱ *Estimated time: 3 days* · Phase 2

Your own portfolio website. Multi-page, real content, deployed to Vercel.

⚡ **New Concepts:** Next.js App Router, file-based routing, layouts, server vs client components, `next/link`, `next/image`, `next/font`

🟣 Warm-up

Topic	What to practise
Next.js App Router — folder structure	Create routes for `/about`, `/projects`, `/contact` by creating folders with `page.tsx` inside. Understand: `app/page.tsx` = `/`, `app/about/page.tsx` = `/about`.
Server vs client components	Create one server component that reads from a file and one client component with `useState`. Add `"use client"` only when you need interactivity.
Shared layouts with `layout.tsx`	Add a `layout.tsx` in `app/` root with a Navbar and Footer. Understand: layout wraps all child pages and does not re-render on navigation.
Tailwind CSS utility classes	Build a responsive card component using only Tailwind. Stack vertically on mobile, side-by-side on `md` screens.
`next/link` and `next/image`	Replace all `<a>` tags with `<Link>`. Replace all `<img>` tags with `<Image>`. Understand: performance, lazy loading, automatic sizing.

Topic	What to practise
`next/font` — optimised fonts *(NEW)*	Load a Google Font using `next/font/google`. Understand why this prevents Cumulative Layout Shift (CLS) caused by font swapping. Never load fonts via a `<link>` tag in Next.js — `next/font` inlines them at build time so the browser never makes a separate request.
Error boundaries — three layers	(1) `error.tsx` catches runtime errors in a route segment; (2) React `<ErrorBoundary>` wraps a subtree in a client component; (3) `try/catch` in server actions catches errors before they propagate. Create a component that throws intentionally and observe each layer.
`next.config.js` — images and domains	Configure `remotePatterns` in `next.config.js` to allow external image domains. Understand what `next/image` requires for remote images and how to set up redirects and rewrites.

⚠️ Non-Negotiable

- Add `not-found.tsx` — shown for any unmatched URL
- Add `error.tsx` — shown when a page throws
- Add at least one React `<ErrorBoundary>` wrapping a non-critical widget
- Configure `next.config.js` with any external image domains you use
- Load all fonts via `next/font` — never via a `<link>` tag

✅ Requirements

- [ ] Home, About, Projects, Contact pages
- [ ] Shared Navbar and Footer via root `layout.tsx`
- [ ] Project cards on the Projects page (hardcoded data is fine)
- [ ] Responsive design with Tailwind — works on mobile and desktop
- [ ] `not-found.tsx` and `error.tsx` boundary pages
- [ ] Deployed to Vercel (free tier)
- [ ] `next.config.js` configured correctly for any external images
- [ ] Fonts loaded via `next/font` — Lighthouse Accessibility score ⩾ 90 after deploy

✦ Stretch Goal

Add a blog section using `.md` files and dynamic routes `/blog/[slug]`.

✓ Ready When

- You can explain the difference between a server and client component and when to use each
- You can explain what `layout.tsx` does and why it does not re-render on navigation

- You can explain why `next/font` prevents CLS and how it differs from a `<link>` tag

---

Project 7 — Movie / TV Show Browser

> ⏱️ *Estimated time: 4 days* · Phase 2

Browse and search movies using the TMDB public API. First real async data project in Next.js.

⚡ **New Concepts:** Server-side fetch in server components, dynamic routes, URL search params, `loading.tsx`, `error.tsx`, env vars

🟣 Warm-up

Topic	What to practise
Fetching in server components	Fetch from `https://jsonplaceholder.typicode.com/posts` directly inside a server component using `async/await` — no `useEffect` needed.
Dynamic routes — `[id]` folders	Create `app/posts/[id]/page.tsx` and use `params.id` to show different content per route.
URL search params for state	Read a search query from the URL using `searchParams` in a server component. A page that filters a list based on `?category=food` in the URL.
`loading.tsx` and Suspense	Add a `loading.tsx` next to a `page.tsx` and see it render during navigation. Understand how Next.js uses React Suspense under the hood.
React Hook Form — basic usage	Build a search form with React Hook Form. Understand: `register`, `handleSubmit`, `formState.errors`. Validate that the search field is not empty.
React Suspense & streaming	Wrap an async server component in `<Suspense fallback={<Skeleton/>}>`. Understand: Suspense lets Next.js stream the page shell immediately and stream in async content as it resolves. Build a page with 2 independent data fetches each in their own `<Suspense>` so they stream independently.

⚠️ Security

Store TMDB API key in `.env.local` as `TMDB_API_KEY`. Only access it in server components — **never** reference in client components.

✅ Requirements

- [ ] Home page shows popular movies — server fetch in server component
- [ ] Click a movie → detail page at `/movies/[id]` with full info
- [ ] Search bar updates results via URL search params (not just state)

- [ ] Loading skeleton (`loading.tsx` or Suspense) while data fetches
- [ ] Error state (`error.tsx`) if the API fails
- [ ] Use React Hook Form for the search input
- [ ] Use `<Suspense>` to independently stream at least one section of the detail page

✦ Stretch Goal

Add a favourites list saved to localStorage — note the client/server split this requires.

✓ Ready When

- You can explain the difference between server fetch and client fetch, and when to choose each
- You understand why the API key must not reach the browser
- You can explain what React Suspense does and how Next.js uses it for streaming

---

Project 7.5 — Notes App with Rich Text ⭐ *Gap-Filler*

> ⏱ *Estimated time: 3 days* · Phase 2

> **Why this exists:** Bridges simple CRUD and the Blog CMS in Phase 3. Teaches Tiptap, debouncing, and the client/server split in practice.

A note-taking app with rich text editing.

⚡ **New Concepts:** Rich text editor (Tiptap), client vs server components in practice, localStorage persistence, debouncing

🟣 Warm-up

Topic	What to practise
When to use client components in Next.js	List 5 scenarios where you MUST use `"use client"`: event handlers, `useState`, `useEffect`, browser APIs, third-party UI libraries.
Third-party components in Next.js	Install `react-hot-toast`. Understand how to wrap third-party providers in a client component and import them in your layout.
Tiptap rich text editor basics	Install `@tiptap/react` and render a basic editor. Add bold, italic, and heading buttons to the toolbar.
Debouncing	Use `lodash.debounce` or a custom `useDebouncedValue` hook to delay a save by 500ms. Build a text input that logs its value 500ms after the user stops typing.

✅ Requirements

- [ ] Create, edit, and delete notes
- [ ] Notes have a title and rich text body (use Tiptap)
- [ ] Toolbar: bold, italic, heading, bullet list
- [ ] Notes list in a sidebar, selected note shown in main area
- [ ] Auto-save to localStorage as the user types (debounced — not on every keystroke)
- [ ] Search/filter notes by title

✦ Stretch Goal

Add tags to notes and filter by tag.

✓ Ready When

- You can explain why a rich text editor requires `"use client"`
- You understand debouncing and why you would not save on every keystroke

---

Project 8 — Dashboard UI

> ⏱ *Estimated time: 4 days* · Phase 2

A data dashboard with charts, stats cards, and a sortable data table. No backend — hardcoded data.

⚡ **New Concepts:** Complex layouts, Recharts, sortable/filterable table, responsive sidebar, Zod for data validation, `useMemo`, `useCallback`, `useTransition`

🟣 Warm-up

Topic	What to practise
Complex layouts with Tailwind — sidebar + main	Build a two-column layout with a fixed-width sidebar and flexible main area. Collapse to single column on mobile.
Recharts — LineChart and BarChart	Render a `LineChart` with hardcoded data. Understand: `ResponsiveContainer`, `XAxis`, `YAxis`, `Tooltip`, `Legend`. Then render a `BarChart` showing 6 months of revenue.
Sortable table	Given an array of objects, implement sort by a clicked column header. Store `{ column, direction }` in state.
Zod — schema definition and parse	Define a Zod schema for a user object and use `z.parse()` to validate it. Understand what happens when data does not match the schema.

Topic	What to practise
`useMemo` and `useCallback` — when and why	Wrap sort + filter logic in `useMemo` so the table does not re-sort on every unrelated re-render. Wrap a handler passed as a prop in `useCallback` so a `React.memo` child does not re-render unnecessarily. **Rule:** only add when you have measured a problem — premature optimisation is a bug.
`useTransition` — keeping the UI responsive *(NEW)*	Understand that `useTransition` marks a state update as non-urgent, so React can keep the UI responsive while a slow update (like filtering a large list) processes in the background. Build a text input that filters a list of 10,000 items — without `useTransition` the input lags; with it, typing stays instant. Understand: `isPending` can drive a loading indicator while the transition runs.

✅ Requirements

- [ ] Sidebar navigation with multiple sections — collapsible on mobile
- [ ] Stats cards: at least 4 (users, revenue, active sessions, churn rate)
- [ ] At least one line chart and one bar chart using Recharts
- [ ] Data table with sort by column (clicking header toggles asc/desc) and text search filter
- [ ] Zod schema defined for all data shapes; `z.array(schema).parse(data)` at startup
- [ ] Fully mobile responsive
- [ ] `useMemo` wrapping the sort + filter computation
- [ ] Table search filter wrapped in `useTransition` so typing never blocks the UI

✦ Stretch Goal

Add a date range picker that filters both charts and table simultaneously.

✓ Ready When

- You can build a complex multi-section layout without fighting CSS
- You can explain the difference between `useMemo` and `useCallback` and state a rule for when NOT to use them
- You can explain what `useTransition` does and give a concrete example of when to use it

---

Project 9 — E-commerce Store

> ⏱ *Estimated time: 5 days* · Phase 2

Product listing, cart, and checkout with form validation. No backend — hardcoded products.

⚡ **New Concepts:** React Context for global state, URL-based filter state, React Hook Form + Zod full validation, derived cart totals

🟣 Warm-up

Topic	What to practise
React Context — `createContext`, `Provider`, `useContext`	Build a `CartContext` that stores items and exposes add/remove/clear. Wrap the app in `CartProvider`. Consume in both the product listing and the navbar.
URL-based filter state with Next.js `searchParams`	Add `?category=food&maxPrice=50` to a URL and read both in a server component. Filters survive page refresh because they live in the URL.
React Hook Form + Zod — full form validation	Build a checkout form with 5 fields. Define a Zod schema. Use `zodResolver` from `@hookform/resolvers`. Show inline validation errors under each field.
Derived state at scale	Given a cart array of `{ product, quantity }`, derive: total items, subtotal, tax, grand total — all computed, never stored.

⚠️ Form Validation Requirements

- Validate: name (required), email (valid format), address (required), card number (exactly 16 digits)
- Show inline error message under each invalid field
- Disable submit button while form is invalid

✅ Requirements

- [ ] Product listing with filter by category and price range
- [ ] Product detail page with image, description, add-to-cart button
- [ ] Add to cart, remove, update quantity
- [ ] Cart total is derived — never stored in state
- [ ] Cart lives in React Context so navbar shows item count
- [ ] Active filters reflected in the URL — page is shareable/bookmarkable
- [ ] Checkout form with React Hook Form + Zod validation

✦ Stretch Goal

Add product comparison — select up to 3 products and compare specs side by side.

✓ Ready When

- You can explain why the cart lives in Context and not in a single component's `useState`
- You can explain the difference between Zod schema validation and manual if/else checks

---

Project 9C — Music / Podcast Library 🔄 *Phase 2 Consolidation*

> ⏱️ *Estimated time: 4 days* · Phase 2

A media library app that uses everything from Phase 2 together. Server fetches, client interactivity, context, forms, responsive layout.

⚡ **New Concepts:** ALL Phase 2 concepts — App Router, server fetch, dynamic routes, Context, React Hook Form + Zod, Recharts, Tailwind layouts

Self-Check Before Starting

> Answer these without looking:
>
> - Which directive makes a component client-only?
> - Can a server component import a client component?
> - Can a client component import a server component?
> - Can you fetch in a server component without `useEffect`?
> - What does `next.config.js` control?
>
> If unsure, revisit Projects 6–7.

✅ Requirements

- [ ] Browse albums/podcasts from iTunes Search API (free, no key required)
- [ ] Dynamic route for each item: `/albums/[id]`
- [ ] Search bar with URL search params
- [ ] Favourites list stored in Context + localStorage
- [ ] A "listening stats" page with at least one Recharts chart (e.g. genres)
- [ ] A "submit a recommendation" form using React Hook Form + Zod
- [ ] Fully responsive with Tailwind
- [ ] `loading.tsx` and `error.tsx` on all data routes

✦ Stretch Goal

Add a playlist feature — drag to reorder tracks (HTML5 drag API).

✓ Ready When

- You can build a complete multi-page Next.js app with server and client components confidently
- You are ready for Phase 3 — databases, auth, and full-stack logic

---

Phase 3 — Full Stack

> 🎯 **Goal:** Prisma · PostgreSQL · NextAuth · Server Actions · TanStack Query · Role-based access
> 🛠️ **Stack:** Next.js App Router + Prisma + PostgreSQL (SQLite for local dev) + NextAuth.js + TanStack Query
> 🔑 **Key Shift:** You now think about the full request lifecycle and data ownership. Every operation must be verified server-side.

Project 10 — Full Stack Task Manager *(Project 1 rebuilt)*

> ⏱️ *Estimated time: 5 days* · Phase 3

Rebuild your Task Manager with a real database and user accounts. Each user sees only their own tasks.

⚡ **New Concepts:** Prisma ORM, PostgreSQL, Next.js API routes, server actions, NextAuth.js sessions, TanStack Query

🟣 Warm-up

Topic	What to practise
Prisma ORM — schema, migrate, query	Define a `User` and `Task` model with a relation in `schema.prisma`. Run `npx prisma migrate dev`. Use `prisma.task.findMany()`, `create()`, `update()`, `delete()` in a test script.
`prisma migrate deploy` vs `migrate dev`	Understand the difference between `prisma migrate dev` (development) and `prisma migrate deploy` (production/CI). Add `"npx prisma migrate deploy"` to your Vercel build command. **Never run `migrate dev` in production.**
Database seeding	Create a `prisma/seed.ts` file that inserts test data. Add `"prisma": { "seed": "ts-node prisma/seed.ts" }` to `package.json`. Run `npx prisma db seed`. Understand why seeding is essential for reproducible dev environments.
NextAuth.js — sessions and providers	Set up NextAuth with GitHub OAuth. Read the session in a server component with `getServerSession()`. Understand: JWT vs database sessions.
Server actions in Next.js	Write a server action that creates a DB record and revalidates the page. Understand `"use server"`, how to call it from a client component, how it differs from an API route.
`useActionState` — consuming server action results *(NEW)*	Use the `useActionState` hook (React 19) to handle the result of a server action directly in the UI. Write a server action that returns `{ success: true } | { success: false; error: string }`. Use

Topic	What to practise
	`useActionState` to show the error inline without a separate `useState`. Understand: this replaces the old `useFormState` from React DOM.
TanStack Query — `useQuery` and `useMutation`	Fetch a list with `useQuery` and display loading/error/data states. Use `useMutation` to create an item and invalidate the query cache on success.
Optimistic updates with TanStack Query	Implement a task toggle using `useMutation` with `onMutate` (update cache immediately), `onError` (roll back), and `onSettled` (always re-fetch). You are ready when you can explain why you need all three callbacks.
Protecting routes and verifying sessions server-side	Write middleware that redirects unauthenticated users to `/login`. Verify session inside a server action before any DB call. Never trust the client.
`server-only` — preventing secret leaks *(NEW)*	Install the `server-only` package. Add `import "server-only"` at the top of any module that contains secrets or DB calls. Understand: if a client component ever imports this module (even indirectly), the build fails with a clear error — catching the leak at build time instead of silently sending secrets to the browser.

⚠️ Security — Non-Negotiable

- Always verify the session server-side before any database read or write
- Never expose another user's data — filter every query by the authenticated user's ID
- Never trust client-sent IDs — verify ownership in every mutation
- **`prisma migrate deploy` in production — never `prisma migrate dev`**
- `.env.local` only for local dev. Set all variables in Vercel dashboard scoped to Development / Preview / Production
- Variables starting with `NEXT_PUBLIC_` are bundled into the client — never put secrets there
- Add `import "server-only"` to every lib file that touches the database or reads secret env vars

✅ Requirements

- [ ] Sign up and log in (NextAuth with GitHub OAuth or email/password)
- [ ] Tasks saved to PostgreSQL via Prisma (not localStorage)
- [ ] Each user sees ONLY their own tasks
- [ ] Create, complete, delete tasks
- [ ] Protected routes — unauthenticated users redirected to `/login`
- [ ] Use TanStack Query for all client-side data fetching

- [ ] Server action responses handled with `useActionState` — errors shown inline
- [ ] `prisma/seed.ts` with test data (at least 2 test users, 5 tasks each)
- [ ] `server-only` added to all lib files that access the DB or secrets
- [ ] Vercel deployment with correct environment variable scoping

✦ Stretch Goal

Add due dates and email reminders using Resend (free tier — 3,000 emails/month).

✓ Ready When

- You understand the full request lifecycle: browser → Next.js server → database → response
- You can explain what TanStack Query's `useQuery` does that `useEffect + fetch` does not
- You can explain the difference between `prisma migrate dev` and `prisma migrate deploy`
- You can explain what `server-only` does and why it catches secret leaks at build time
- You can explain what `useActionState` does and how it differs from handling server action results manually with `useState`

Project 10.5 — Link Shortener ⭐ *Gap-Filler*

> ⏱ *Estimated time: 2 days* · Phase 3

> **Why this exists:** Short, focused project to get comfortable with server actions, database writes, and redirects before tackling the Blog CMS.

Paste a long URL and get a short link. Track click counts.

⚡ **New Concepts:** Server actions, `prisma.create`/`findUnique`, Next.js `redirect()`, `nanoid` for slug generation, rate limiting

🟣 Warm-up

Topic	What to practise
Next.js `redirect()` in server actions	Call `redirect('/dashboard')` inside a server action after a successful DB write. Understand: `redirect()` throws internally — do not catch it.
Generating unique slugs	Use the `nanoid` library to generate a 6-character random string. Understand collision probability and how to handle it with a `try/catch` on unique constraint violations.
Prisma — `findUnique` and	`findUnique` by a non-ID field (e.g. slug). Write an `upsert` that creates if not exists or updates if it does.

Topic	What to practise
`upsert`	
Rate limiting	Install `@upstash/ratelimit` (free tier). Add rate limiting to your link creation endpoint: max 10 links per user per hour. Return a 429 response with a user-friendly message when exceeded. Understand why rate limiting belongs on the server, not the client.

✅ Requirements

- [ ] Paste a long URL and get a short link (e.g. `yourapp.com/abc123`)
- [ ] Visiting the short link redirects to the original URL
- [ ] Track click count per link (increment on each visit)
- [ ] User must be logged in to create links
- [ ] User can view and delete their own links
- [ ] Stats table: link, original URL, clicks, created date
- [ ] Rate limiting: max 10 links created per user per hour

✦ Stretch Goal

Let the user choose a custom slug instead of auto-generating.

✓ Ready When

- You can write a server action that reads from a form, writes to the DB, and redirects
- You can explain why rate limiting is server-side and how to implement it

---

Project 11 — Blog with CMS

> ⏱ *Estimated time: 6 days* · Phase 3

An admin writes and publishes posts. Public visitors read them. Role-based access control.

⚡ **New Concepts:** Role-based access (admin vs public), Tiptap rich text, image uploads, slug routing, typed server action responses

🟣 Warm-up

Topic	What to practise
Role-based access control in Next.js	Extend the NextAuth session to include a `role` field (admin/user). Check the role in middleware to protect `/admin/*` routes. Check the role again in every server action that modifies data — not just in the UI.

Topic	What to practise
Image uploads to Cloudinary or Vercel Blob	Upload a file from a form using a server action. Store the returned URL in the database. Display it with `next/image`.
Typed server action responses	Write a server action returning `{ success: true, data: Post }` or `{ success: false, error: string }` — never throws to the client. Handle both cases in the UI.
Slug-based routing and uniqueness	Generate a slug from a title using `slugify`. Ensure slugs are unique in the DB with a Prisma unique constraint.
Sentry error monitoring	Install `@sentry/nextjs` and run `npx @sentry/wizard`. Add a `Sentry.captureException(error)` call in at least one server action `catch` block. Verify the error appears in the Sentry dashboard.

✅ Requirements

- [ ] Public: browse published posts, read individual posts at `/blog/[slug]`
- [ ] Admin: log in, access `/admin` dashboard, create/edit/publish/delete posts
- [ ] Posts: title, rich text content (Tiptap), cover image, slug, published/draft status
- [ ] Images uploaded to Cloudinary or Vercel Blob
- [ ] Role enforced server-side in middleware AND in every server action
- [ ] Every server action returns a typed success/error response — no unhandled throws
- [ ] Sentry installed and capturing errors in production

✦ Stretch Goal

Add comments — authenticated users can comment, admin can delete any comment.

✓ Ready When

- You can explain how role-based access is enforced at the server level, not just in the UI
- You can explain why you check the role in the server action even though the route is already protected
- You have Sentry set up and can explain what it monitors

Project 12 — SaaS Starter — Team Project Manager

> ⏱ *Estimated time: 7 days* · Phase 3

Users create workspaces, invite members, manage projects and tasks together. Multi-tenant architecture.

⚡ **New Concepts:** Multi-tenancy, workspace/member data model, relational data at scale, invitation flow, Stripe billing

🟣 Warm-up

Topic	What to practise
Multi-tenancy data modelling	Design a schema with User, Workspace, Membership (join table with role), Project, Task. Every query must be scoped to a workspace. Membership verified on every request.
Invitation flow — email + token	Generate a signed invitation token (use `jose` or `jsonwebtoken`), email it with Resend, validate it when the recipient clicks the link.
Prisma relations and nested queries	Use Prisma's `include` to fetch a workspace with its members and projects in one query. Understand the N+1 query problem and how `include` solves it.
Permission checks at every layer	Write a helper `isWorkspaceMember(userId, workspaceId)` that queries the DB. Call it at the start of every server action touching workspace data.
Stripe billing integration	Set up a Stripe account. Implement a "free plan: max 3 projects" limit checked server-side before project creation. Add a Stripe Checkout session for upgrading to paid. Handle the `checkout.session.completed` webhook to update the workspace plan in the DB.
Webhook security — signature verification and idempotency *(NEW)*	Stripe sends webhooks that can be replayed (network retries, infrastructure hiccups). Two problems to solve: (1) **Verify the signature** — use `stripe.webhooks.constructEvent(body, sig, secret)` with your webhook signing secret; reject any request where this throws. Never process an unverified event. (2) **Idempotency** — store processed event IDs in the DB (`stripeEventId String @unique`). Before handling an event, check if you've seen that ID before; if yes, return 200 immediately. This prevents double-charging or double-granting access when the same event arrives twice. Use `ngrok` (free) to receive Stripe webhooks on localhost during development.

⚠️ Database Schema to Design

```
User        — id, email, name, passwordHash
Workspace   — id, name, ownerId, createdAt, plan (free/pro)
Membership  — userId, workspaceId, role (owner/member), joinedAt
Project     — id, workspaceId, name, description
Task        — id, projectId, assigneeId, title, status, dueDate
StripeEvent — id, stripeEventId (unique), processedAt  ← idempotency table
```

✅ Requirements

- [ ] Sign up and create a workspace
- [ ] Invite users by email — they receive a link to join
- [ ] Create projects inside a workspace
- [ ] Create and assign tasks inside a project
- [ ] Members only see workspaces they belong to
- [ ] Roles: owner (full access) and member (read + create tasks, no delete workspace)
- [ ] All data scoped to workspace — no cross-workspace data leaks
- [ ] Free plan: max 3 projects per workspace (enforced server-side)
- [ ] Stripe Checkout for upgrading to paid plan (unlimited projects)
- [ ] Stripe webhook handler for `checkout.session.completed`
- [ ] Webhook signature verified on every incoming event
- [ ] Processed event IDs stored in DB — duplicate events ignored safely

✦ Stretch Goal

Add a Stripe customer portal so users can manage their billing and cancel subscriptions.

✓ Ready When

- You can explain how you scoped every database query to a workspace
- You can draw the full Prisma schema from memory
- You can explain the Stripe payment flow end to end
- You can explain why webhook idempotency matters and how you implemented it

---

Project 12C — Real-time Chat App 🔁 *Phase 3 Consolidation*

> ⏱ *Estimated time: 5 days* · Phase 3

A real-time chat app with rooms and auth. Adds one new concept (WebSockets via Pusher) while consolidating all full-stack patterns.

⚡ **New Concepts:** Pusher Channels for real-time, all Phase 3 patterns: auth, DB, server actions, multi-user data, `useOptimistic`

🟣 Warm-up

Topic	What to practise
Pusher Channels — subscribe and publish	Set up a free Pusher account. Subscribe to a channel in a client component and trigger an event from a server action. See the message arrive in real time.

Topic	What to practise
Review: auth + DB + server actions	Review your Project 10 or 11 code before starting. Auth is not new, DB writes are not new — only the real-time layer is new.
`useOptimistic` — React 19	Use the `useOptimistic` hook to show a sent message immediately in the UI before the server confirms it. Understand: `useOptimistic` gives you a temporary optimistic state that is rolled back if the server action fails. This is the React 19 alternative to TanStack Query's `onMutate` pattern.
`useOptimistic` + Server Action — the modern mutation pattern *(NEW)*	Combine `useOptimistic` directly with a Server Action (no TanStack Query needed for mutations). The pattern: call `addOptimistic(newItem)` to update the UI immediately, then `await yourServerAction(data)` in the same handler. If the action fails, the optimistic state rolls back automatically. Build a simple todo list using only this pattern — no `useMutation`, no manual cache invalidation. Understand when you'd still choose TanStack Query (complex cache dependencies, background refetch) vs this pattern (simple optimistic mutations).

✅ Requirements

- [ ] Auth with NextAuth (reuse patterns from Project 10)
- [ ] Create and join chat rooms
- [ ] Send and receive messages in real time using Pusher (free tier) or Ably
- [ ] Messages persisted to PostgreSQL — history loads on join
- [ ] Show who is online in a room (presence channel)
- [ ] User can only send messages if authenticated
- [ ] Messages appear instantly using `useOptimistic` before server confirms
- [ ] At least one mutation implemented with `useOptimistic + Server Action` directly (not TanStack Query)

✦ Stretch Goal

Add message reactions (emoji) — real-time updates for all room members.

✓ Ready When

- You can build a full-stack Next.js app with auth, database, and real-time features
- You can explain `useOptimistic` and when to use it instead of TanStack Query's `onMutate`
- You can explain when you'd use the `useOptimistic + Server Action` pattern vs TanStack Query
- You are ready for Phase 4 — testing, CI, and production-level practices

---

Phase 4 — Production Level

> 🎯 **Goal:** Testing · CI/CD · Performance · Accessibility · Monitoring · Real users · Ship it
> 🛠️ **Stack:** Everything from Phase 3 + Vitest + React Testing Library + Playwright + GitHub Actions + Sentry
> 🔑 **Key Shift:** You now think about reliability, maintainability, monitoring, and what happens when real users break things.

Project 13 — Add Full Test Coverage to a Previous Project

> ⏱ *Estimated time: 7 days* · Phase 4

Pick Project 10, 11, or 12. Add unit, component, and end-to-end tests. Add a CI pipeline.

⚡ **New Concepts:** Vitest, React Testing Library, Playwright, GitHub Actions CI, MSW, testing philosophy

🟣 Warm-up

Topic	What to practise
Testing philosophy — behaviour not implementation	Write a test for a form component that fills in fields and clicks submit, then asserts the success state. Do NOT test that `setState` was called. Test what the user sees, not how the code works internally.
Vitest — unit tests for pure functions	Write unit tests for 3 utility functions (e.g. `formatCurrency`, `slugify`, `calculateTotal`). Understand: `describe`, `it`/`test`, `expect`, `toBe`, `toEqual`.
React Testing Library — render and interact	Render a `TaskList` with mock data. Use `getByRole`, `getByText`, `userEvent.click`. Assert that clicking Delete removes the item.
Playwright — end-to-end browser tests	Write a test that opens the app, fills in the login form, clicks Login, and asserts the dashboard is visible.
GitHub Actions — CI pipeline basics	Create `.github/workflows/ci.yml` that runs `npm test` on every push. Understand: triggers, jobs, steps, `actions/checkout`, `actions/setup-node`.
MSW (Mock Service Worker)	Install `msw`. Intercept a TanStack Query fetch in tests and return mock data. Understand why mocking at the network level is better than mocking the `fetch` function directly. Write a component test where the API is mocked to return an error and assert the error UI is shown.
`vi.mock` — mocking modules	Use `vi.mock` to mock specific modules in Vitest. The most important case in Next.js projects: `vi.mock('next/navigation', () => ({ useRouter: ()`

Topic	What to practise
and Next.js internals *(NEW)*	`=> ({ push: vi.fn() }), redirect: vi.fn() }))`. Without this, any component that calls `useRouter` or `redirect` will throw in tests. Also mock `next/headers` for server components that read cookies or headers. Write a component test that verifies a button calls `router.push('/dashboard')` after a successful form submit.

Testing Philosophy

> **Good test:** fill form → click submit → assert item appears in list
> **Bad test:** assert that `setState` was called with a specific value
> Ask: "What would a real user do?" — test that, not the internals.

✅ Requirements

- [ ] Unit tests for all utility/helper functions
- [ ] Unit tests for all `useReducer` reducers
- [ ] Component tests for key interactions: form submit, add item, delete item, filter
- [ ] At least one MSW mock for an API call — test both success and error paths
- [ ] `next/navigation` mocked with `vi.mock` in all component tests that use routing
- [ ] At least one end-to-end test with Playwright: full login flow
- [ ] GitHub Actions CI pipeline — tests run on every push, PR blocked if tests fail
- [ ] Test coverage report generated (`vitest --coverage`)

✦ Stretch Goal

Write tests before writing the component (TDD) for at least one new feature.

✓ Ready When

- You can write a component test before writing the component itself
- You can explain the difference between a unit test, integration test, and end-to-end test
- You can mock an API call using MSW and test both success and error paths
- You can mock `next/navigation` with `vi.mock` and explain why it's necessary

Project 13.5 — Performance & Accessibility Audit ⭐ *Gap-Filler*

> ⏱ *Estimated time: 3 days* · Phase 4

Audit and fix one of your previous projects for Core Web Vitals, Lighthouse score, and WCAG accessibility. Not a new build — a deep improvement sprint.

⚡ **New Concepts:** Lighthouse, Core Web Vitals (LCP/CLS/FID), WCAG 2.1 AA, `next/image`, font optimisation, bundle analysis, Sentry performance, feature flags

🟣 Warm-up

Topic	What to practise
Core Web Vitals — LCP, CLS, FID/INP	Run a Lighthouse audit on your portfolio site. Understand what each score means. Identify the top 3 issues it flags.
WCAG 2.1 AA accessibility rules	Use the axe DevTools browser extension on your Movie Browser project. Fix every error it reports. Understand: colour contrast, focus indicators, ARIA roles, skip links.
Next.js bundle analysis	Install `@next/bundle-analyzer` and run `ANALYZE=true npm run build`. Identify the largest packages in your bundle.
`next/image` — advanced optimisation *(NEW)*	Beyond basic usage: use the `sizes` prop to tell the browser which image size to load at each viewport width (e.g. `sizes="(max-width: 768px) 100vw, 50vw"`). Add `priority` to above-the-fold images so they are preloaded. Add `placeholder="blur"` with `blurDataURL` for a low-quality placeholder while the image loads. Each of these directly improves LCP and CLS — verify the improvement in Lighthouse.
Sentry performance monitoring	Enable Sentry performance tracing (`tracesSampleRate: 0.2`). View a transaction in the Sentry dashboard. Identify the slowest DB query in a Phase 3 project and add a database index to fix it.
Feature flags with environment variables	Implement a simple feature flag using an environment variable (`NEXT_PUBLIC_FEATURE_NEW_UI=true`). Wrap a new UI section so it only renders when the flag is enabled. Understand why feature flags let you deploy code without activating features.

✅ Requirements

- [ ] Pick any one project from Phase 2 or 3
- [ ] Run Lighthouse audit — document scores before and after
- [ ] Achieve Lighthouse Performance score ⩾ 90 on desktop
- [ ] Fix all critical axe accessibility errors (0 critical errors)
- [ ] Achieve Lighthouse Accessibility score ⩾ 90
- [ ] Add skip-to-content link for keyboard users
- [ ] Ensure all interactive elements have visible focus indicators
- [ ] All fonts loaded via `next/font` — confirm no CLS from font swap in Lighthouse
- [ ] At least one `next/image` with correct `sizes`, `priority`, and `placeholder` props
- [ ] Add Sentry performance tracing and identify + fix at least one slow query
- [ ] Implement one feature behind a feature flag
- [ ] Write a short 1-page audit report: what you found, what you fixed, scores before/after

✦ Stretch Goal

Achieve a perfect 100 Lighthouse score across all 4 categories.

✓ Ready When

- You can run a Lighthouse audit and understand every metric
- You can identify and fix common accessibility violations
- You can explain what a feature flag is and why teams use them
- You can use Sentry to find and fix slow database queries
- You can explain the `sizes`, `priority`, and `placeholder` props on `next/image` and what each one improves

---

Project 14 — Your Own Real Product

> ⏱ *Your timeline* · Phase 4

Build something you would actually use or sell. Idea is yours. Apply everything from the roadmap.

⚡ **New Concepts:** Product thinking, real users, every skill from the roadmap — you decide the stack

How to Pick Your Idea

> Find one thing you do manually that a simple web app could handle.
> Build for one specific type of person.
> It does not need to be original — a better, simpler version of something that exists is a great product.
> **Avoid:** social network, marketplace, "Uber for X" — too big and too vague.
> **Scope your MVP:** Write down the 3 things your product MUST do to be useful. Everything else is a stretch goal. Build the 3 things first.

✅ Requirements

- [ ] Deployed and publicly accessible (Vercel is fine)
- [ ] Real users — even just 5 friends or family members using it counts
- [ ] At least one paid feature or a waiting list (Stripe or Lemon Squeezy)
- [ ] Sentry installed and capturing production errors
- [ ] Rate limiting on all mutation endpoints
- [ ] You can talk through every technical decision you made and why
- [ ] `README.md` documents: what it does, how to run it locally, architecture decisions

✦ Stretch Goal

Write a public case study — what you built, decisions made, what you'd do differently. Publish it on your portfolio.

✓ Ready When

This is your portfolio centrepiece — there is no next step except to keep building.

Complete Technology Checklist

> Use this before job applications to verify you've touched every key technology.

React Core

- [ ] `useState`, `useReducer`, `useRef`, `useEffect`
- [ ] `useMemo`, `useCallback`
- [ ] `useTransition` / `startTransition`
- [ ] `useContext`, custom hooks
- [ ] `useContext + useReducer` combined pattern
- [ ] `useOptimistic` (React 19)
- [ ] `useActionState` (React 19)
- [ ] Component composition, props, derived state
- [ ] Controlled forms
- [ ] Error boundaries (`error.tsx` + React `<ErrorBoundary>` + `try/catch`)
- [ ] React Suspense and streaming (wrapping async components)

TypeScript

- [ ] Interfaces and type aliases
- [ ] Generics (basic)
- [ ] Discriminated unions
- [ ] Typing props, state, reducers, context values

Next.js

- [ ] App Router + file-based routing
- [ ] Server vs client components
- [ ] Dynamic routes `[id]`
- [ ] Layouts, `loading.tsx`, `error.tsx`, `not-found.tsx`
- [ ] Server actions + API routes
- [ ] `next/image` (including `sizes`, `priority`, `placeholder`)
- [ ] `next/link`
- [ ] `next/font`

- [ ] URL search params
- [ ] Environment variables (local / preview / production scoping)
- [ ] `next.config.js` (image domains, redirects, rewrites)
- [ ] Streaming with Suspense

Data & Forms

- [ ] React Hook Form
- [ ] Zod validation
- [ ] TanStack Query (`useQuery`, `useMutation`, optimistic updates)
- [ ] Prisma ORM
- [ ] PostgreSQL basics
- [ ] `prisma migrate dev` vs `prisma migrate deploy`
- [ ] Database seeding (`prisma/seed.ts`)
- [ ] Debouncing (`useDebouncedValue` / `lodash.debounce`)

Auth & Security

- [ ] NextAuth.js sessions
- [ ] Protected routes + middleware
- [ ] Server-side session verification
- [ ] Role-based access control
- [ ] Environment secret management
- [ ] `server-only` package (build-time secret leak prevention)
- [ ] Rate limiting (Upstash or equivalent)

Billing & Payments

- [ ] Stripe Checkout session
- [ ] Stripe webhook handler
- [ ] Webhook signature verification
- [ ] Webhook idempotency (deduplicating replayed events)
- [ ] Plan enforcement server-side
- [ ] Stripe customer portal (stretch)

Testing & Production

- [ ] Vitest unit tests
- [ ] React Testing Library
- [ ] MSW (Mock Service Worker) for API mocking in tests
- [ ] `vi.mock` for module mocking (especially `next/navigation`)
- [ ] Playwright e2e tests
- [ ] GitHub Actions CI
- [ ] Vercel deployment

- [ ] Lighthouse / Core Web Vitals
- [ ] Accessibility (WCAG / axe)
- [ ] Git workflow + meaningful commits
- [ ] Bundle analysis (`@next/bundle-analyzer`)

Monitoring & Observability

- [ ] Sentry error monitoring (`@sentry/nextjs`)
- [ ] Sentry performance tracing
- [ ] Feature flags via environment variables
- [ ] Structured error responses from server actions

Project Log

#	Project	Started	Finished	Days	Repo	Key Lesson
1	Task Manager					
2	Expense Tracker					
2.5	Weather Widget ⭐					
3	Quiz App					
4	Recipe Book					
5	Kanban Board					
5.5	Theme & Settings App ⭐					
5C	Habit Tracker 🔁					
6	Portfolio Site					
7	Movie Browser					
7.5	Notes App ⭐					
8	Dashboard UI					
9	E-commerce Store					
9C	Music Library 🔁					
10	Full Stack Tasks					
10.5	Link Shortener ⭐					

#	Project	Started	Finished	Days	Repo	Key Lesson
11	Blog CMS					
12	SaaS Starter					
12C	Chat App 🔁					
13	Test Coverage					
13.5	Perf & A11y Audit ⭐					
14	Your Own Product					

> ⭐ = Gap-filler 🔁 = Consolidation
> *v4.1 — Six targeted additions to v4: `useTransition` (Project 8), `next/font` (Project 6), `server-only` package (Project 10), `useActionState` (Project 10), webhook signature verification + idempotency (Project 12), `vi.mock` for Next.js internals (Project 13), advanced `next/image` props (Project 13.5), `useOptimistic + Server Action` pattern (Project 12C).*